Data Management Mini Toolkit

BST, Graph Traversal & Hash Table — Analysis Report

Data Structures | ETCCDS202 | Unit 4
School of Engineering & Technology, K.R. Mangalam University

Student: Kinshuk | Roll No.: 2501730171
Program: B.Tech CSE (AI/ML) | Semester: 2

April 2025

1. Binary Search Tree (BST)

1.1 How BST Works

A Binary Search Tree is a binary tree where every node satisfies: all keys in the left subtree are smaller than the node's key, and all keys in the right subtree are larger. This property makes search, insert, and delete efficient without scanning every element.

1.2 Why Inorder Traversal Gives Sorted Output

Inorder traversal visits nodes in the order: **Left → Root → Right**. Because of the BST property, the leftmost node is always the smallest key, and the rightmost is the largest. By going left first at every level before visiting the root and then the right subtree, we naturally visit keys in ascending order. For the tree built from [50, 30, 70, 20, 40, 60, 80], inorder gives [20, 30, 40, 50, 60, 70, 80] — perfectly sorted.

1.3 Delete — Three Cases

Deletion in a BST must preserve the BST property. There are three cases:

```
Case 1 - Leaf node (no children): Simply remove the node. Example: delete 20.
Case 2 - One child: Replace the node with its only child. Example: delete 60
(which had only right child 65). Case 3 - Two children: Find the inorder
successor (smallest node in right subtree), copy its key into the current node,
then delete the successor. Example: delete 30 (inorder successor is 40).
```

1.4 Time Complexity

Operation	Average Case	Worst Case	Notes
insert(key)	O(log n)	O(n)	Worst when tree is skewed (like a linked list)
search(key)	O(log n)	O(n)	Same reasoning as insert
delete(key)	O(log n)	O(n)	Finding successor also takes O(log n) average
inorder()	O(n)	O(n)	Must visit every node once

Average O(log n): In a balanced BST, each comparison eliminates half the remaining nodes — similar to binary search. **Worst O(n):** If you insert keys in sorted order (e.g., 10, 20, 30, 40...), every new node goes to the right and the tree becomes a straight chain — searching it is no better than a linked list. This is why balanced BSTs (AVL, Red-Black trees) are used in practice.

2. Graph — BFS and DFS

2.1 Graph Representation

The graph is stored as an **adjacency list**: a dictionary where each key is a node and its value is a list of (neighbour, weight) tuples. This uses less memory than an adjacency matrix for sparse graphs and makes neighbour lookup straightforward.

```
Graph used (directed, weighted): A -> B(2), C(4) B -> D(7), E(3) C -> E(1), F(8)
D -> F(5) E -> D(2), F(6) F -> (no outgoing edges)
```

2.2 BFS vs DFS — Simple Explanation

	BFS (Breadth-First Search)	DFS (Depth-First Search)
Strategy	E plore all neighbours first, then their neighbour	Go as deep as possible before backtracking
Data structure	Queue (FIFO)	Stack / Recursion
Result from A	A → B → C → D → E → F	A → B → D → F → E → C
Pattern	Level by level (wide)	Branch by branch (deep)

2.3 Real-World Use Cases

BFS: Finding the shortest path in an unweighted graph (e.g., minimum number of hops between two routers in a network, or the fewest moves to solve a puzzle). Since BFS explores level by level, the first time it reaches the destination is guaranteed to be via the shortest path.

DFS: Cycle detection in a graph (e.g., detecting circular dependencies in a package manager), topological sorting of tasks, and exploring all possible paths in a maze or decision tree.

2.4 Time Complexity

Algorithm	Time Complexity	Space Complexity	Notes
BFS	O(V + E)	O(V)	V = nodes, E = edges; visits each once
DFS	O(V + E)	O(V)	F cursion stack depth is O(V) worst cas

3. Hash Table — Separate Chaining

3.1 What is a Collision and Why it Happens

A hash function maps a key to a bucket index. With a table of size 5 and the function **index = key % 5**, multiple keys can map to the same index. For example: 10 % 5 = 0, 15 % 5 = 0, 20 % 5 = 0 — three different keys, same bucket. This is called a **collision**. Collisions are unavoidable when the number of possible keys is larger than the table size.

3.2 Collision Demonstration

```
Table size = 5, hash function: key % 5 insert(10) -> 10 % 5 = 0 -> Bucket[0]:
[(10, 'ten')] insert(15) -> 15 % 5 = 0 -> Bucket[0]: [(10,'ten'),
(15,'fifteen')] COLLISION insert(20) -> 20 % 5 = 0 -> Bucket[0]: [(10,'ten'),
(15,'fifteen'), (20,'twenty')] COLLISION insert(7) -> 7 % 5 = 2 -> Bucket[2]:
[(7, 'seven')] insert(12) -> 12 % 5 = 2 -> Bucket[2]: [(7,'seven'),
(12,'twelve')] COLLISION
```

3.3 Why Separate Chaining Helps

In separate chaining, each bucket holds a **linked list (or Python list)** of all key-value pairs that hash to that index. Instead of rejecting a key when the bucket is occupied, we simply append it to the chain. To retrieve a key, we hash it to find the bucket, then scan the short chain for the exact key. To delete, we find and remove the matching entry from the chain without affecting other entries in the same bucket.

After deleting key 15 from Bucket[0], the chain becomes [(10, 'ten'), (20, 'twenty')] — key 10 and key 20 are still accessible correctly.

3.4 Time Complexity

Operation	Average Case	Worst Case	Notes
insert(key, value)	O(1)	O(n)	O(1) if load factor is low; O(n) if all keys collide
get(key)	O(1)	O(n)	Sca chain in bucket; chain length is O(1) on aver ge
delete(key)	O(1)	O(n)	Same as get, then remove the entry

Average O(1): With a good hash function and a reasonable load factor (number of keys / table size), chains stay short — typically 1–2 entries. This makes all operations near-constant time. **Worst O(n):** If all n keys hash to the same bucket (e.g., all multiples of table size), one chain holds all n entries and every operation scans the full chain. This is why table resizing (rehashing) is done in practice when load factor grows too high.

4. Summary

Module	Structure Used	Key Operations	Avg Time	Real-World Use
BST	Binary Tree	insert, search, delete, inorder	O(log n)	Databases, sorted sets

Graph	Adjacency List	BFS, DFS	O(V+E)	Routing, social networks
Hash Table	Array + Chains	insert, get, delete	O(1)	Caching, hash maps

5. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. [Chapters 12 (BST), 22 (Graph), 11 (Hash Tables)]

2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley.

3. GeeksforGeeks — BST operations, BFS, DFS, Hashing articles. https://www.geeksforgeeks.org/

4. Course notes: Data Structures (ETCCDS202), K.R. Mangalam University, Unit 4.